

Stage G5b — Semantic Cache: Production Architecture Review

Context: Multi-tenant business assistant. Answers depend on `(business_id)`, `(locale)`, a per-business knowledge base, dynamic business data, tools, and agent reasoning. **Goal:** Cut LLM generation calls by 40–60% without degrading answer quality, while preventing cross-business contamination, wrong semantic matches, stale answers, hallucinated hits, and cache poisoning.

o. The core failure your draft must design around

Embedding only the raw query is insufficient for three independent reasons:

1. **Semantic proximity $\neq$ answer equivalence.** Embeddings under-weight exactly the tokens that decide a business answer: numbers, entity/plan names, negation, and modifiers (`(annual)` vs `(monthly)`, `(cancel)` vs `(pause)`, `(EU)` vs `(US)`). "Cost of the Pro plan" and "cost of the Enterprise plan" embed at very high cosine but have different answers. This is the canonical **false-positive / hallucinated hit**, and it cannot be removed by threshold tuning alone — raising the threshold to exclude it also removes legitimate paraphrase hits (the **false-negative** side). You need a second signal.
2. **A query embedding does not encode the answer's dependencies.** The same query has different correct answers across time (price changed), KB versions (policy updated), and users (entitlement scope). A query-keyed cache is blind to all of this, which is the root cause of stale answers and cross-context bleed.
3. **A cache must store *why* an answer was valid, not just the answer.** Without provenance (which KB version, which data snapshot, which retrieval set), you cannot decide when the entry has gone stale.

The design below addresses each: **canonicalize** (fixes recall), **key/filter on dependencies** (fixes contamination + staleness), **verify borderline hits** (fixes hallucinated hits), **gate write-back** (fixes poisoning), **classify volatility** (routes uncacheable queries away from the cache).

1. How leading systems implement semantic caching

The 7 dimensions you asked for, split across two tables for readability, then notes on the instructive cases.

1a. Lookup / write-back / invalidation

System / pattern	Cache lookup flow	Write-back strategy	Invalidation
OpenAI ecosystem	Native = <i>prefix</i> (KV) caching, automatic, prefix-prefix match. Semantic response caching is app-built (e.g. via GPTCache wrapping the client).	Prefix cache: implicit. Semantic: app writes embedding+response on miss.	Prefix cache: short TTL (~5 min, refresh on hit). Semantic: app-managed.
Anthropic agent pattern	<i>Prefix</i> caching via explicit <code>cache_control</code> breakpoints on stable system prompt + tool schemas; agent loop reuses the prefix across turns.	Implicit cache write on first call; ephemeral.	TTL ~5 min refreshed on hit; 1-hour tier available. Not semantic — it cheapens calls, doesn't skip them.
LangChain	Native caches (<code>InMemoryCache</code> , <code>SQLiteCache</code> , <code>RedisCache</code>) are exact-match only . Semantic comes via GPTCache or Redis as a vector store.	Exact caches store full I/O keyed by exact prompt. Semantic delegated to the backend.	Exact: manual/TTL. Semantic: backend-dependent.
Redis (RedisVL <code>SemanticCache</code> / LangCache)	Embed prompt → HNSW ANN over a Redis index → return if within <code>distance_threshold</code> (COSINE distance, 0–2, lower = stricter). Supports <code>filterable_fields</code> for metadata pre-filter.	Write embedding + response + metadata into the same Redis index on miss. Managed LangCache automates embedding + eviction + metrics.	Per-key TTL; LRU/LFU eviction; explicit delete; filter-based scoping.
RAG production systems	Two sub-patterns: (a) cache final response keyed by query; (b) retrieval-aware — key includes the retrieved chunk set so a hit requires the same grounding.	Write after generation; better systems also persist the retrieval signature.	Naturally invalidated when retrieval changes; otherwise TTL + re-index events.

System / pattern	Cache lookup flow	Write-back strategy	Invalidation
Customer-support AI	Heavy use of exact + semantic over an FAQ/intent taxonomy; paraphrases map to one entry.	Write-back gated to non-personalized, resolved answers; curated "golden" FAQ entries seeded offline.	TTL by topic volatility + editorial purge when policy changes.
Enterprise copilots	Multi-layer: semantic cache for the generic answer, then a personalization/augmentation step (cheap model + user context + live RAG) on top of the cached core.	Cache only the generic, non-personalized core.	Version + event-driven; per-document ACL refresh on permission-change webhooks.
Multi-tenant platforms	Filtered ANN scoped to the tenant, or per-tenant namespace/index routed at query time.	Per-tenant write path; never write across tenant boundary.	Per-tenant TTL + targeted purge; tenant offboarding = namespace delete.

1b. Tenant isolation / threshold / failure modes & tradeoffs

System / pattern	Tenant isolation	Similarity threshold	Key failure mode & tradeoff
OpenAI / Anthropic prefix cache	Per-account; not a multi-tenant <i>response</i> cache.	N/A (exact prefix).	A single dynamic token (timestamp, user id, stray newline) in the prefix drops hit rate to ~0. Tradeoff: huge savings on stable prefixes, none on variable ones.
LangChain native	None built-in for semantic.	Exact only.	Near-zero hit rate on natural-language variation. Tradeoff: trivial to add, but only

System / pattern	Tenant isolation	Similarity threshold	Key failure mode & tradeoff
			useful for dev/exact use cases.
Redis RedisVL / LangCache	<code>filterable_fields</code> (Tag/Numeric) for <code>business_id</code> , <code>locale</code> ; separate logical DBs or key prefixes.	Single <code>distance_threshold</code> per cache instance; you tune it.	Static global threshold is blunt across heterogeneous intents. Tradeoff: sub-ms lookups and one platform for vectors+KV+TTL vs. you own threshold calibration.
GPTCache	Metadata + collection scoping.	Configurable similarity evaluator + threshold; static, needs manual per-domain tuning.	Documented limits: static thresholds, no partial cache update when documents change (staleness), response-storage memory overhead. Project no longer adds new model APIs.
RAG (retrieval- aware)	Per-tenant index/namespace.	Query threshold + retrieval-signature match as a second gate.	You must run retrieval before checking the cache, so retrieval cost is not saved. Tradeoff: you only save generation (the dominant cost) but get near-automatic staleness handling.
Customer support	Per-tenant FAQ space.	High threshold per intent; confusable intents separated by intent label.	Over-broad matching across similar intents. Tradeoff: high hit rate on a narrow domain vs. brittle on long-tail.

System / pattern	Tenant isolation	Similarity threshold	Key failure mode & tradeoff
Enterprise copilots	Namespace/index per tenant; ACLs mirrored into chunk metadata.	High threshold on the generic core; personalization layer absorbs variation.	The cheap personalization model can still introduce errors/cost; only helps when a generic core answer exists.
Multi-tenant platforms	Namespace-per-tenant (pool) as default; dedicated index (silo) for large/high-compliance tenants. Scope enforced at the query layer from a signed identity claim — never from app code that can forget the filter.	Per-tenant or per-intent calibrated.	The classic leak is an omitted tenant filter on a shared index → another tenant's neighbors surface. Tradeoff: pool = cheap/scalable, silo = strongest isolation/cost.

Notes on the instructive cases

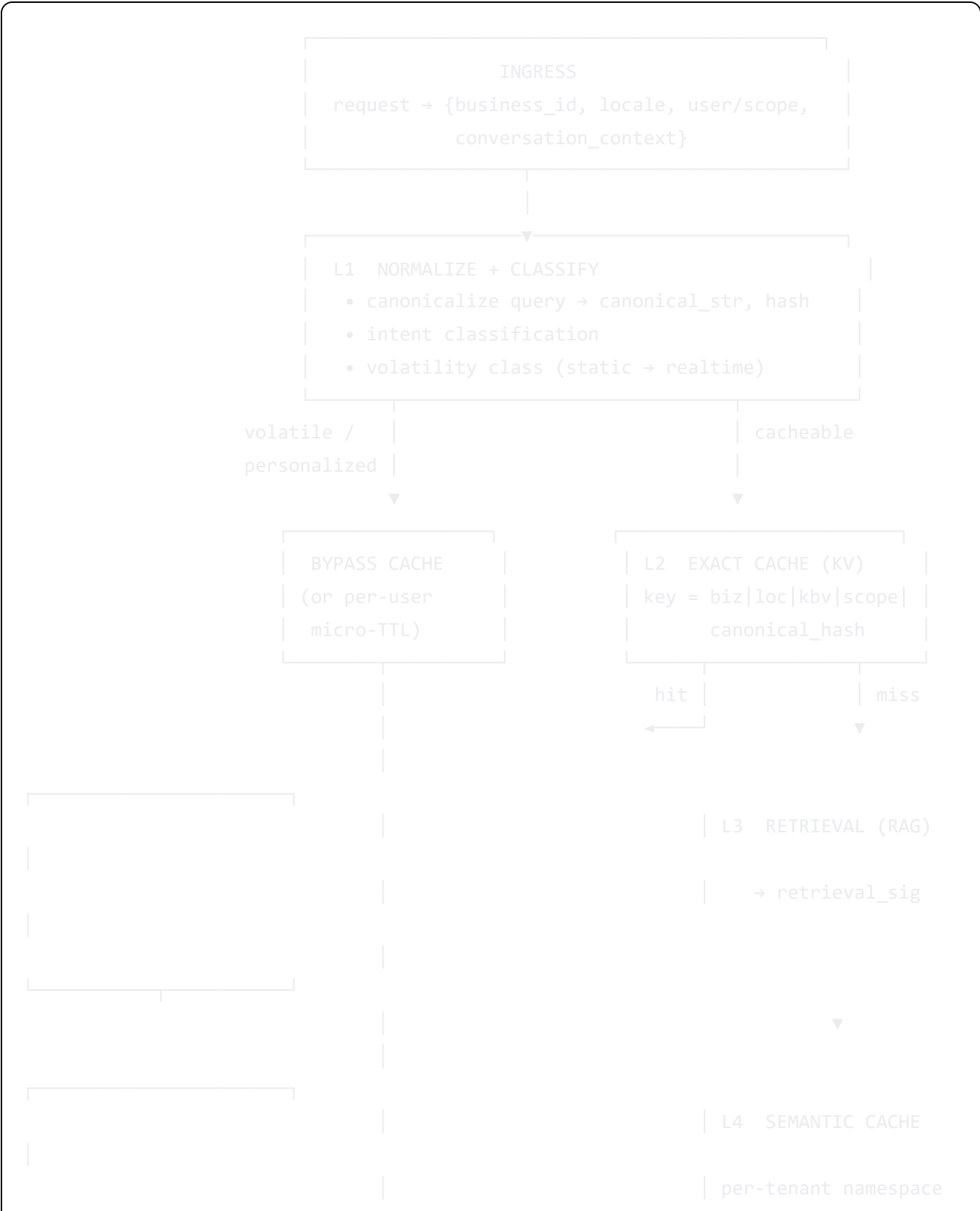
- **Prefix caching is complementary, not competing.** Use semantic caching to skip generation; on the miss path, prefix-cache the static system prompt + tool schemas so the unavoidable generation is cheaper. This is the "semantic → prefix → inference" tiering several teams report.
- **GPTCache's documented limitations are your spec in negative form.** Static thresholds, no invalidation when documents change, and memory overhead are precisely the three things your design must beat — which is why per-domain thresholds, retrieval-aware keying, and gated write-back are non-negotiable.
- **The "context-enabled" copilot pattern is a real option for your dynamic-data problem.** Cache only a *generic* core answer semantically, then run a cheap model that fuses the cached core with live business data + user context. You get a semantic hit on the stable part and only pay a small model to personalize, instead of full generation. Treat this as a phase-2 lever for personalized/semi-dynamic intents (caveats: the cheap fuser adds cost and a small error surface; it only helps where a generic core exists).

2. Evaluating your alternatives A–H (verdict for *your* use case)

Option	Verdict	Why / how to use it
A. Query-only embeddings	Reject as primary	Highest hallucinated-hit risk (the plan-tier problem). Keep only as one component <i>inside</i> a guarded design, never as the sole match signal.
B. Intent-aware embeddings	Adopt partially	Don't embed <i>only</i> the intent (loses specificity). Use a lightweight intent classifier for two things: routing (volatile vs cacheable) and as a cache-key dimension that separates confusable intents (<code>(cancel)</code> vs <code>(pause)</code>).
C. Query + business context	Adopt as filter, not as text	<code>(business_id)</code> / <code>(locale)</code> belong in metadata filters / namespaces, not concatenated into the embedded text (embeddings don't treat IDs meaningfully). Optionally prepend the business <i>domain</i> in natural language only if you share one index across tenants — but namespacing is cleaner.
D. Canonicalized query	Adopt	Normalize (lowercase, strip filler, expand abbreviations, resolve coreference; optional cheap-LLM rewrite to a canonical question). Raises hit rate and enables an exact layer via the canonical hash.
E. Retrieval-aware caching	Adopt (high value)	Key the entry on the retrieval signature (sorted <code>(chunk_id@version)</code>). A hit requires query similarity and the same grounding. This is the cleanest staleness defense you have. Cost: retrieval runs before the cache check — acceptable, since generation is the expensive step.
F. Response caching	Adopt, enriched	This is the artifact you store — but persist provenance (KB version, data version, retrieval signature, volatility class), not just the text.
G. Agent-step caching	Adopt selectively (phase 2)	Memoize expensive <i>deterministic</i> sub-steps (tool calls, retrieval, classification) so even response-cache misses get partial savings. More plumbing; add after the response cache is stable.
H. Hybrid exact + semantic	Adopt as the backbone	L1 exact (zero false positives, serves the hot head cheaply) + L2 semantic (paraphrase coverage) + verification on the gray zone. This is the production standard.

Net design = D + E + F + H as the core, B + C as routing/keying, G as a later optimization, and the context-enabled fuser as an optional lever for personalized intents.

3. Recommended architecture (component view)





		quality gate: confident? grounded? not refusal/"I don't know"?
		deterministic? no PII/personalization? cacheable
volatility?		→ write L2 (exact) + L4 (semantic) with full provenance
metadata		

CROSS-CUTTING - L7 Invalidation: version bump • retrieval-sig mismatch • event purge • TTL

- Observability: hit-rate (exact/semantic), sampled false-hit eval, per-tenant metrics

4. Request flow (step sequence)

1. Receive request. Resolve trusted identity → business_id, locale, entitlement_scope
(from a signed claim, NOT from the request body).
2. L1: canonicalize query → canonical_str + canonical_hash; classify intent + volatility.
3. IF volatility ∈ {realtime, personalized-transactional}:
→ skip cache, go to step 7 (optionally a per-user micro-TTL cache only).
4. L2 EXACT: GET key =
business_id|locale|kb_version|entitlement_scope|canonical_hash
→ HIT ⇒ return cached answer. [O(1), zero false-positive] — exits here
→ MISS ⇒ continue.
5. L3 RETRIEVAL: run the agent's normal RAG retrieval → retrieval_signature
= sorted(chunk_id@chunk_version).
6. L4 SEMANTIC: embed(canonical_str) → filtered ANN within the tenant namespace,
filter on locale + kb_version, top-k.
 - a. best candidate similarity ≥ τ_{high} AND retrieval_signature matches
⇒ return cached answer. — exits here
 - b. candidate in [τ_{low} , τ_{high}) OR signature mismatch
⇒ VERIFY (cross-encoder or cheap-LLM "same question, same answer given this grounding?"):
pass ⇒ return cached answer. — exits here
fail ⇒ continue.
 - c. best candidate < τ_{low} ⇒ continue.
7. L5 AGENT: run full pipeline (tools + reasoning + generation).

```
Prefix-cache the static system prompt + tool schemas to cheapen this call.
8. Return the freshly generated answer to the user immediately.
9. L6 WRITE-BACK (async, does not block the response):
    IF passes quality gate (confident, grounded, not a refusal/IDK,
        deterministic, no PII/personalization, cacheable volatility):
        write to L2 (exact) and L4 (semantic) with provenance + TTL.
```

Two latency notes: keep **verification only for the gray zone** (most hits are clean auto-accepts), and run the semantic lookup **in parallel** with the start of retrieval so the extra embed+ANN doesn't serialize onto the miss path.

5. Database schema (cache entry record)

The authoritative record for each cached answer (relational or document store; the vector lives in the vector index, referenced by `cache_id`).

Field	Type	Purpose
<code>cache_id</code>	UUID (PK)	Entry id; also the vector record id.
<code>business_id</code>	string (indexed)	Tenant. Drives namespace + filter.
<code>locale</code>	string (indexed)	e.g. <code>ro-RO</code> , <code>en-US</code> . Filter dimension.
<code>intent</code>	string	Intent label from L1; separates confusable intents.
<code>volatility_class</code>	enum	<code>static</code> <code>semi_dynamic</code> <code>dynamic</code> <code>realtime</code> . Drives TTL/cacheability.
<code>canonical_query</code>	text	Normalized query (what was embedded).
<code>canonical_hash</code>	string (indexed)	Hash of <code>canonical_query</code> ; powers the exact layer.
<code>answer</code>	text	The cached response.
<code>answer_format</code>	enum	<code>markdown</code> <code>text</code> <code>json</code> (tool-shaped answers).
<code>retrieval_signature</code>	string[]	Sorted <code>chunk_id@version</code> ; the grounding fingerprint.
<code>kb_version</code>	int / string	Knowledge-base version at write time.
<code>data_version</code>	int / string	Dynamic-data snapshot version (if applicable).
<code>entitlement_scope</code>	string	Scope hash if answers vary by plan/role; else <code>public</code> .
<code>embedding_model</code>	string	Model + version that produced the vector (space identity).
<code>quality_score</code>	float	Gate score at write time (confidence/groundedness).
<code>model</code>	string	Generating model/version (provenance).
<code>source_request_id</code>	string	Trace back to the originating request.
<code>created_at</code>	timestamp	Write time.
<code>last_hit_at</code>	timestamp	For LRU/refresh logic.
<code>hit_count</code>	int	Popularity (LFU, analytics).
<code>ttl_expires_at</code>	timestamp	Hard expiry (by volatility class).

Field	Type	Purpose
<code>is_curated</code>	bool	"Golden" answer; TTL-exempt, explicit-invalidate only.

6. Vector schema

```

index:                one logical index, partitioned by namespace = business_id
                      (dedicated index for large / high-compliance tenants)
vector field:         embedding[d]                # d = embedding model dimension
distance metric:      COSINE
ANN algorithm:        HNSW    (M ≈ 16-32, ef_construction ≈ 128-256, ef_runtime tuned for
recall)
record id:            cache_id
filterable fields (pre-filter, applied during the ANN scan):
    business_id       TAG                # primary tenant boundary (or use namespace)
    locale             TAG
    kb_version         TAG/NUM
    volatility_class   TAG
    entitlement_scope  TAG
payload (returned with the hit): cache_id, answer, retrieval_signature, kb_version,
quality_score

```

Notes: use **pre-filtering** (filter inside the ANN scan), not post-filtering — post-filtering a shared index can return top-k that are all from other tenants, hurting recall and risking leakage. If you use RedisVL's `SemanticCache`, the same idea maps to `filterable_fields` + a `distance_threshold` in COSINE-distance units (0–2, lower = stricter). The embedding-space identity (`embedding_model`) is part of the entry; **changing the embedding model invalidates the whole index** (see §7).

7. Cache invalidation strategy (layered)

Invalidation is your defense against stale and hallucinated answers. Use four mechanisms together, primary first:

1. **Version-keyed (primary, logical).** `kb_version` and `data_version` are part of the exact key and the semantic filter. Bumping a tenant's version makes all older entries *unreachable* instantly and atomically; they are lazily evicted afterward. This is cheap and per-tenant.

2. **Retrieval-signature mismatch (automatic, self-healing).** When a business edits its KB, retrieval for affected queries changes → the stored `retrieval_signature` no longer matches → treated as a miss and regenerated. Covers exactly the subset of answers that depended on changed documents.
3. **Event-driven targeted purge.** On a KB edit or price/policy change webhook: delete entries whose `retrieval_signature` contains the affected `chunk_id`, or purge the whole tenant namespace for large changes.
4. **TTL by volatility (backstop).** Bounds staleness even if no event fires (see §8).

`is_curated` entries are exempt from TTL and invalidated only by explicit editorial action.

Embedding-model changes are a special case: re-embed the index or invalidate it wholesale, because cross-model cosine values are not comparable.

8. TTL strategy (tiered by volatility)

Volatility class	Examples	TTL	Notes
<code>static</code>	Return policy, brand FAQ, "what is X"	Days–weeks, or until version bump	Best hit-rate tier. Seed curated entries here offline.
<code>semi_dynamic</code>	Feature availability, plan names	Hours–1 day	Re-validate against version on hit.
<code>dynamic</code>	Prices, holiday hours, stock levels	Minutes, or do not cache	Prefer the context-enabled fuser (cache generic core, inject live value).
<code>realtime / personalized</code>	Order status, account balance, "my ..."	Never cache (or per-user, seconds TTL, isolated)	Route around the cache at L1.

Add LRU/LFU eviction under memory pressure. **Refresh-on-hit** (extend TTL for hot entries) is fine, but cap absolute age (`created_at + max_age`) so a popular entry can't outlive its underlying data indefinitely.

9. Multi-tenant strategy

This is where a contamination bug is unrecoverable, so isolation is enforced structurally, not by convention.

- **Default: namespace-per-tenant (pool pattern).** Queries are scoped to a single namespace; this isolates recall (one tenant's traffic can't surface in another's top-k), keeps per-query cost proportional to that tenant's data, and makes offboarding a single namespace delete.
 - **Silo for large / high-compliance tenants.** A dedicated index/collection for the biggest or most regulated tenants. Document the silo-vs-pool decision for auditors.
 - **Enforce scope at the query layer from a trusted identity.** The `business_id` used for namespace selection and filtering comes from a signed claim (e.g. a JWT claim), **never** from the request body or an app-level metadata field that a code path can forget to set. The single most common multi-tenant leak is an omitted tenant filter on a shared index.
 - **Separate the exact-cache keyspace per tenant** (key prefix or separate logical DB).
 - **No global fallback search.** If a tenant's namespace misses, you regenerate — you never widen the search across tenants.
 - **No shared embeddings, no shared "popular answers" pool** across tenants.
 - **Per-tenant** metrics, quotas/rate limits, and a **purge API**.
 - The context-enabled personalization layer (if used) must read user context from a **per-tenant, per-user** store and must scrub PII before anything is written to a shared tenant cache.
-

10. Cost-saving estimates

Use this model rather than a headline number:

```
monthly_generation_savings
  = requests
  × cacheable_fraction           # share NOT routed away as realtime/personalized
  × effective_hit_rate           # exact-hits + accepted-semantic-hits
  × (cost_per_generation - cost_per_cache_serve)

cost_per_cache_serve ≈ embedding_cost
                    + vector_query_cost
                    + (gray_zone_fraction × verification_model_cost)
```

`cost_per_cache_serve` is tiny next to a full agent generation, so net savings track the **generation-call reduction** almost directly.

What drives `effective_hit_rate`:

- **Traffic repetition.** Support/FAQ traffic is Zipfian — a small head of questions dominates. The exact layer alone often captures that head safely.

- **Canonicalization.** Collapses paraphrases onto shared entries, lifting both exact and semantic hits.
- **Not over-caching volatile intents.** Routing realtime/personalized queries away protects quality and keeps the cached set high-precision.

Reference points from the field (treat as ranges to validate, not promises):

- ~31% of queries are semantically similar to a prior query in studied workloads.
- Reported RAG semantic-cache hit rates span roughly 18–60%.
- Agent *prefix*-caching hit rates often exceed 60% (different mechanism — cheaper calls, not skipped calls).
- Vendor latency claims for cache hits land around 40–50% reduction.

Verdict on your 40–60% generation-call target: plausible for a support-style assistant *if* (a) real query repetition exists, (b) you canonicalize, and (c) you route volatile intents around the cache. Instrument first: ship with logging of `from_cache`, `similarity`, exact-vs-semantic, and per-intent hit rates, then tune. **Stack prefix caching on the miss path** for compounding savings on the generations you can't avoid.

11. Risks and mitigations

Risk	Mitigation
Cross-business contamination	Namespace-per-tenant + scope from signed identity at the query layer + no global fallback search + separate exact keyspace.
Wrong semantic match (false hit)	Conservative <code>τ_high</code> for auto-accept + verification on the gray zone + retrieval-signature match + tighten/force-verify on numeric/entity-bearing queries.
Stale answers	Version-keyed invalidation (primary) + retrieval-signature mismatch + event purge + volatility TTL + never-cache realtime.
Hallucinated cache hit	Same as false-hit controls, plus only cache <i>grounded</i> , <i>gated</i> answers (never cache an ungrounded generation).
Cache poisoning	Async write-back behind a quality gate (confidence, groundedness, not a refusal/IDK, deterministic); quarantine low-confidence; sampled audits; one-click purge; never cache tool errors.
Personalization / PII leak into shared cache	Never cache personalized answers in the shared tenant cache; per-user cache isolated + seconds TTL; scrub PII before any shared write.

Risk	Mitigation
Threshold drift / embedding-model change	<code>embedding_model</code> is part of the entry; re-embed or invalidate on model change; continuous offline false-hit eval recalibrates thresholds per intent.
Cold start (empty cache)	Seed curated/golden FAQ entries offline; backfill from historical logs; accept a lower early hit rate.
Latency regression on the miss path	Verification only in the gray zone; run semantic lookup in parallel with retrieval; budget the lookup; async write-back.

12. Final recommendation

Build the **layered hybrid**: canonicalization + intent/volatility routing in front of a two-tier cache (**L1 exact** for the safe hot head, **L2 semantic** for paraphrase coverage), **retrieval-aware keying** with signature validation for staleness, **gray-zone verification** to kill hallucinated hits, **async gated write-back** to prevent poisoning, **per-tenant namespacing** with identity-enforced scope for isolation, **version-keyed + event-driven invalidation** with tiered TTL, and **prefix caching on the miss path** to compound savings. Adopt your alternatives as **D + E + F + H** at the core, **B + C** for routing/keying, **G** later, and the **context-enabled fuser** as an optional lever for personalized intents.

"If this had to support millions of requests per month, what exact architecture would you deploy and why?"

Exactly the layered hybrid above, with these concrete bindings:

Topology

- **L1 exact cache + hot path in Redis** (RedisVL `SemanticCache` / managed LangCache if you want embedding, eviction, TTL, and metrics handled for you). Redis gives sub-millisecond exact lookups and co-locates vectors, KV, and TTL on one platform.
- **L2 semantic** either in Redis (if Redis-centric) or a managed vector DB with first-class **namespaces** (e.g. Pinecone-style) when tenant fan-out is large and you want per-namespace cost/scaling. **Namespace per tenant**; dedicated index for the largest/most-regulated tenants.
- **Small, fast embedding model** with its own **embedding cache** (never re-embed an identical canonical string), so embedding cost stays flat as traffic grows.
- **A cross-encoder reranker (or cheap-LLM judge)** invoked only on the gray zone.

- **Async write-back** workers off the response path; **event-driven invalidation** consumers on KB/data-change webhooks.
- **Prefix caching** enabled on the agent's static system prompt + tool schemas for the miss path.
- **A continuous offline eval harness** that samples production hits, labels them same-answer/different-answer, and recalibrates per-intent thresholds; **per-tenant dashboards** for hit rate, false-hit rate, latency saved, and staleness incidents.

Why this and not the simpler draft At millions/month, **generation cost dominates** and a **single false hit erodes trust across an entire tenant** — so the system must be **precision-first** (a missed hit just costs a little money; a wrong hit costs credibility). The exact layer serves the Zipfian head safely and cheaply; the semantic layer covers paraphrases; **retrieval-aware keying makes the cache self-invalidate as each business's KB drifts** (the exact gap GPTCache-style designs leave open); **gray-zone verification removes the plan-tier / entity-swap false hits** that pure thresholds can't; **namespacing + identity-enforced scope makes cross-tenant leakage structurally impossible** rather than convention-dependent; and **prefix caching compounds savings on the unavoidable misses**. That combination is what lets you hit 40–60% fewer generation calls *while* holding answer quality and tenant isolation — which is the actual objective, not raw hit rate.